

MIPT & YSDA · AUTUMN 2026

# Deep Generative Models

## Lecture 2

Roman Isachenko

Moscow Institute of Physics and Technology  
Yandex School of Data Analysis

# Recap of Previous Lecture

We're given a **finite** number of i.i.d. samples  $\{\mathbf{x}_i\}_{i=1}^n \subset \mathbb{R}^m$  drawn from an **unknown** distribution  $p_{\text{data}}(\mathbf{x})$ .

## Objective

Our aim is to estimate a distribution  $p_{\text{data}}(\mathbf{x})$  that allows us to:

- **Sampling (generation):** sample  $\mathbf{x} \sim p_{\text{data}}(\mathbf{x})$ .
- **Density evaluation:** evaluate  $p_{\text{data}}(\mathbf{x})$  on novel data ("How likely is an object  $\mathbf{x}$ ?").

## What is a Divergence?

- $D(\pi \| p) \geq 0$  for all  $\pi, p \in \mathcal{P}$
- $D(\pi \| p) = 0$  if and only if  $\pi \equiv p$

## Divergence Minimization Problem

$$\min_{\theta} D(p_{\text{data}} \| p_{\theta})$$

# Recap of Previous Lecture

## Forward KL

$$D_{\text{KL}}(p_{\text{data}} \| p_{\theta}) = \int p_{\text{data}}(\mathbf{x}) \log \frac{p_{\text{data}}(\mathbf{x})}{p_{\theta}(\mathbf{x})} d\mathbf{x} \rightarrow \min_{\theta}$$

## Reverse KL

$$D_{\text{KL}}(p_{\theta} \| p_{\text{data}}) = \int p_{\theta}(\mathbf{x}) \log \frac{p_{\theta}(\mathbf{x})}{p_{\text{data}}(\mathbf{x})} d\mathbf{x} \rightarrow \min_{\theta}$$

## Maximum Likelihood Estimation (MLE)

Let  $\{\mathbf{x}_i\}_{i=1}^n$  be i.i.d. observed samples.

$$\theta^* = \arg \max_{\theta} \prod_{i=1}^n p_{\theta}(\mathbf{x}_i) = \arg \max_{\theta} \sum_{i=1}^n \log p_{\theta}(\mathbf{x}_i).$$

Maximum likelihood estimation is equivalent to minimizing a Monte Carlo estimate of the forward KL divergence.

# Recap of Previous Lecture

## Likelihood as a Product of Conditionals

For  $\mathbf{x} = (x_1, \dots, x_m)$ ,  $\mathbf{x}_{1:j} = (x_1, \dots, x_j)$ ,

$$p_{\theta}(\mathbf{x}) = \prod_{j=1}^m p_{\theta}(x_j | \mathbf{x}_{1:j-1}); \quad \log p_{\theta}(\mathbf{x}) = \sum_{j=1}^m \log p_{\theta}(x_j | \mathbf{x}_{1:j-1})$$

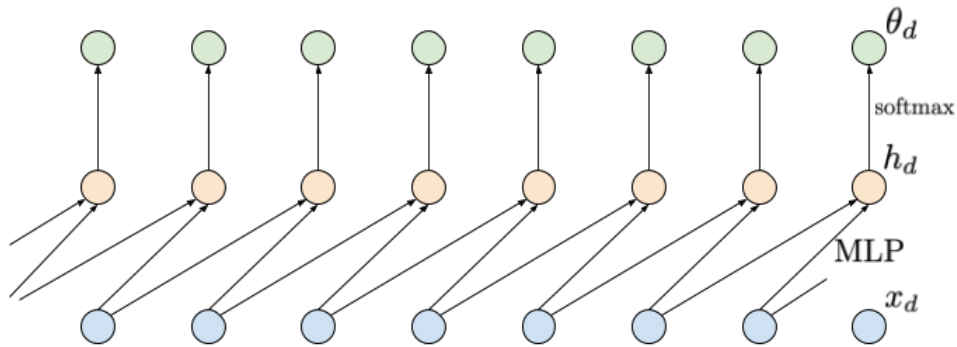
$$\theta^* = \arg \max_{\theta} \sum_{i=1}^n \left[ \sum_{j=1}^m \log p_{\theta}(x_{ij} | \mathbf{x}_{i,1:j-1}) \right]$$

## Sampling (Ancestral)

1. Sample  $\hat{x}_1 \sim p_{\theta}(x_1)$ ,  $\hat{x}_2 \sim p_{\theta}(x_2 | \hat{x}_1)$ ,  $\dots$ ,  $\hat{x}_m \sim p_{\theta}(x_m | \hat{\mathbf{x}}_{1:m-1})$ .
2. Return  $\hat{\mathbf{x}} = (\hat{x}_1, \hat{x}_2, \dots, \hat{x}_m)$ .

# Recap of Previous Lecture

## Autoregressive Models: MLP



## Autoregressive Models: ImageGPT

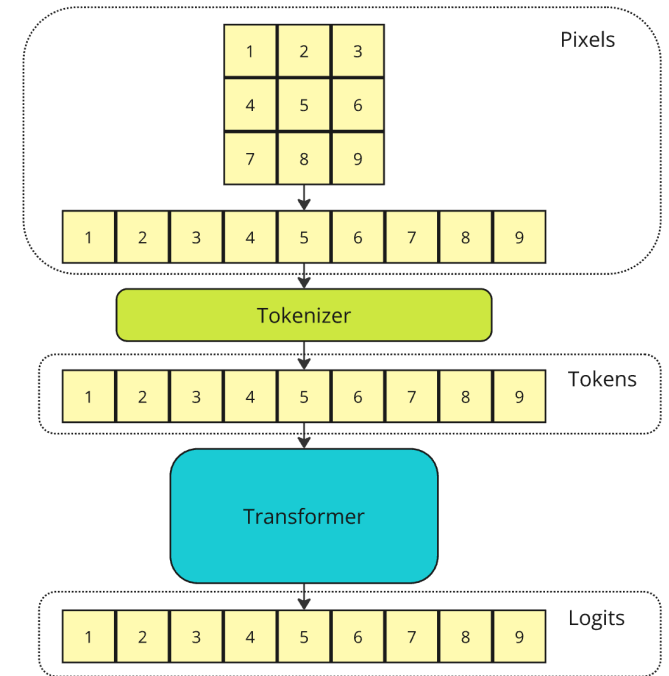
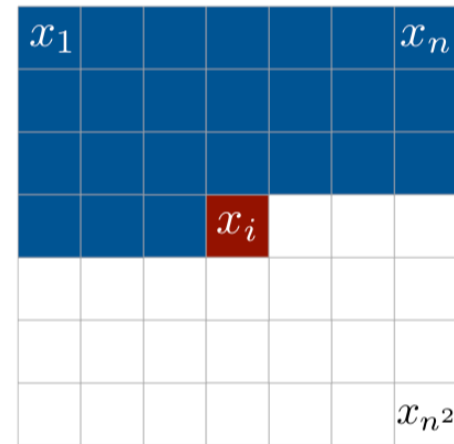


Image credit: [https://jmtomczak.github.io/blog/2/2\\_ARM.html](https://jmtomczak.github.io/blog/2/2_ARM.html)

Oord A., Kalchbrenner N., Kavukcuoglu K. Pixel Recurrent Neural Networks, 2016

Chen M. et al. Generative Pretraining from Pixels, 2020

# Outline

## 01 Normalizing Flows (NF)

## 02 NF Examples

- Linear NF

- Gaussian Autoregressive NF

- Blockwise Flows (RealNVP / TarFlow)

## 03 Latent Variable Models (LVM)

# Outline

## 01 Normalizing Flows (NF)

## 02 NF Examples

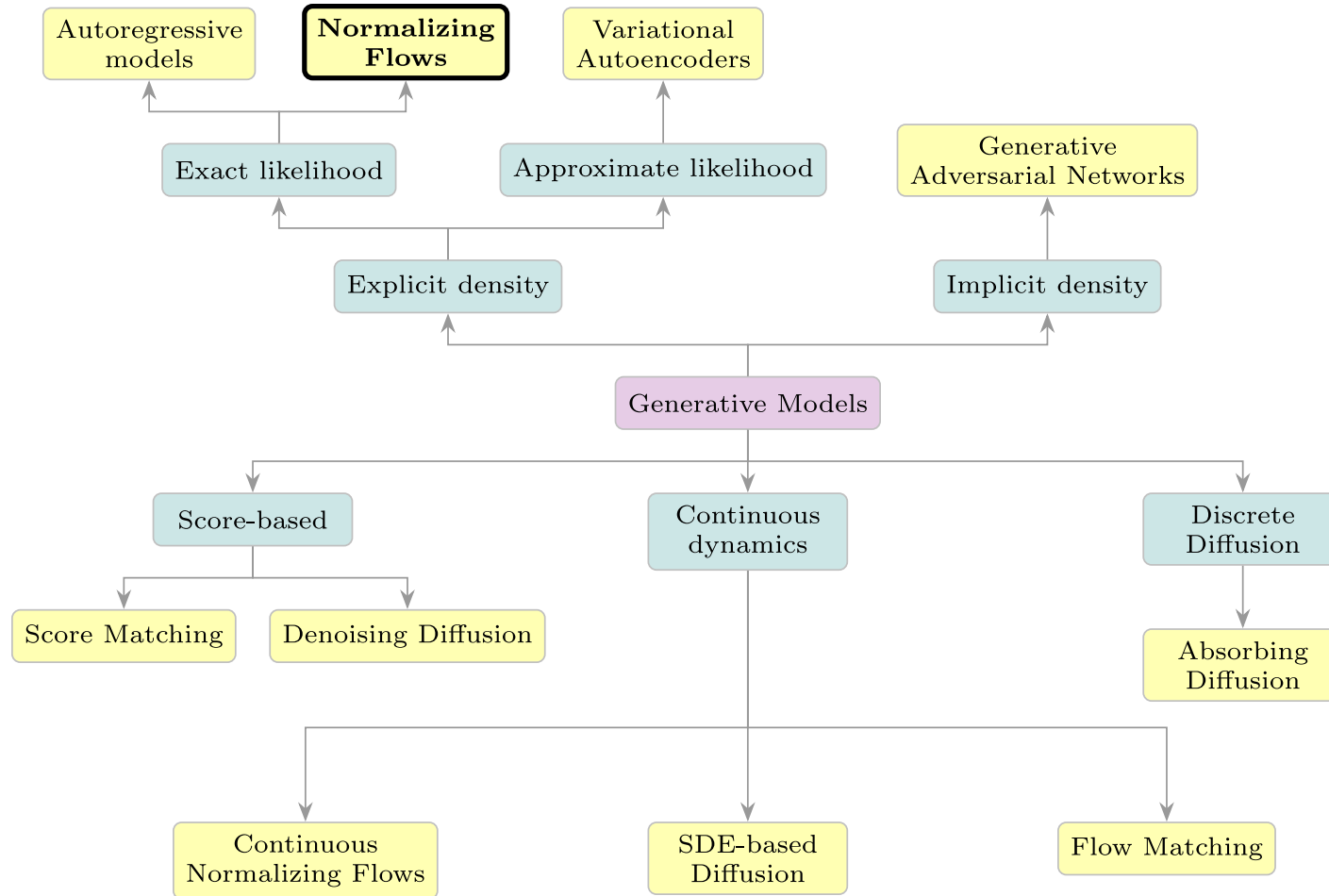
- Linear NF

- Gaussian Autoregressive NF

- Blockwise Flows (RealNVP / TarFlow)

## 03 Latent Variable Models (LVM)

# Generative Models Taxonomy



# Normalizing Flows: Prerequisites

## Jacobian Matrix

Let  $\mathbf{f} : \mathbb{R}^m \rightarrow \mathbb{R}^m$  be a differentiable function.

$$\mathbf{z} = \mathbf{f}(\mathbf{x})$$

$$\mathbf{J} = \frac{\partial \mathbf{z}}{\partial \mathbf{x}} = \begin{pmatrix} \frac{\partial z_1}{\partial x_1} & \cdots & \frac{\partial z_1}{\partial x_m} \\ \vdots & \ddots & \vdots \\ \frac{\partial z_m}{\partial x_1} & \cdots & \frac{\partial z_m}{\partial x_m} \end{pmatrix} \in \mathbb{R}^{m \times m}$$

## Change of Variables Theorem (CoV)

Let  $\mathbf{x} \in \mathbb{R}^m$  be a random vector with density  $p(\mathbf{x})$ , and let  $\mathbf{f} : \mathbb{R}^m \rightarrow \mathbb{R}^m$  be a  $C^1$ -diffeomorphism ( $\mathbf{f}$  and  $\mathbf{f}^{-1}$  are continuously differentiable mappings). If  $\mathbf{z} = \mathbf{f}(\mathbf{x})$ , then

$$p(\mathbf{x}) = p(\mathbf{z}) |\det(\mathbf{J}_{\mathbf{f}})| = p(\mathbf{z}) \left| \det \left( \frac{\partial \mathbf{z}}{\partial \mathbf{x}} \right) \right| = p(\mathbf{f}(\mathbf{x})) \left| \det \left( \frac{\partial \mathbf{f}(\mathbf{x})}{\partial \mathbf{x}} \right) \right|$$

$$p(\mathbf{z}) = p(\mathbf{x}) |\det(\mathbf{J}_{\mathbf{f}^{-1}})| = p(\mathbf{x}) \left| \det \left( \frac{\partial \mathbf{x}}{\partial \mathbf{z}} \right) \right| = p(\mathbf{f}^{-1}(\mathbf{z})) \left| \det \left( \frac{\partial \mathbf{f}^{-1}(\mathbf{z})}{\partial \mathbf{z}} \right) \right|$$

# Jacobian Determinant

## Inverse Function Theorem

If the function  $\mathbf{f}$  is invertible and its Jacobian is continuous and non-singular, then

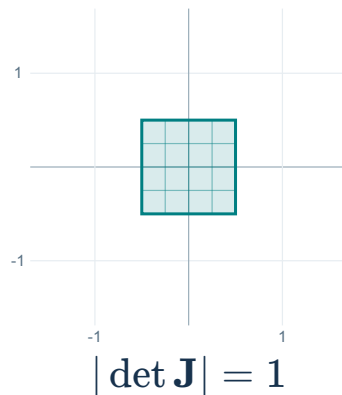
$$\mathbf{J}_{\mathbf{f}^{-1}} = \mathbf{J}_{\mathbf{f}}^{-1}.$$

$$|\det(\mathbf{J}_{\mathbf{f}^{-1}})| = \frac{1}{|\det(\mathbf{J}_{\mathbf{f}})|}.$$

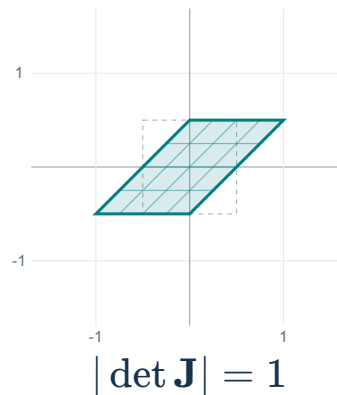
- $\mathbf{x}$  and  $\mathbf{z}$  reside in the same space ( $\mathbb{R}^m$ ).
- $\mathbf{f}_{\theta}(\mathbf{x})$  is a parameterized transformation.

The determinant of the Jacobian  $\mathbf{J} = \frac{\partial \mathbf{f}_{\theta}(\mathbf{x})}{\partial \mathbf{x}}$  quantifies how the volume is changed by the transformation.

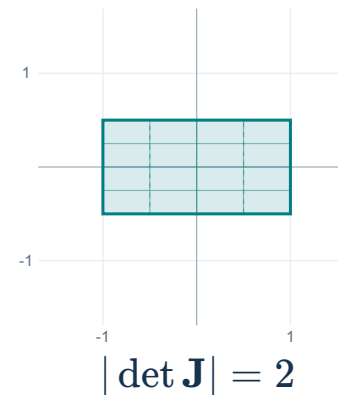
Identity



Shear



Stretch



$$\mathbf{z} = \mathbf{J}\mathbf{x}, \quad \mathbf{J} = \begin{pmatrix} a & b \\ 0 & 1 \end{pmatrix}$$

Shear preserves area; stretching by 2 doubles it.

$$p(\mathbf{z}) = p(\mathbf{x}) / |\det \mathbf{J}| \text{ — probability mass is preserved.}$$

# Fitting Normalizing Flows

## MLE Problem

$$p_{\theta}(\mathbf{x}) = p(\mathbf{z}) \left| \det \left( \frac{\partial \mathbf{z}}{\partial \mathbf{x}} \right) \right| = p(\mathbf{f}_{\theta}(\mathbf{x})) \left| \det \left( \frac{\partial \mathbf{f}_{\theta}(\mathbf{x})}{\partial \mathbf{x}} \right) \right|$$

$$\log p_{\theta}(\mathbf{x}) = \log p(\mathbf{f}_{\theta}(\mathbf{x})) + \log |\det(\mathbf{J}_{\mathbf{f}})| \rightarrow \max_{\theta}$$

### Inference

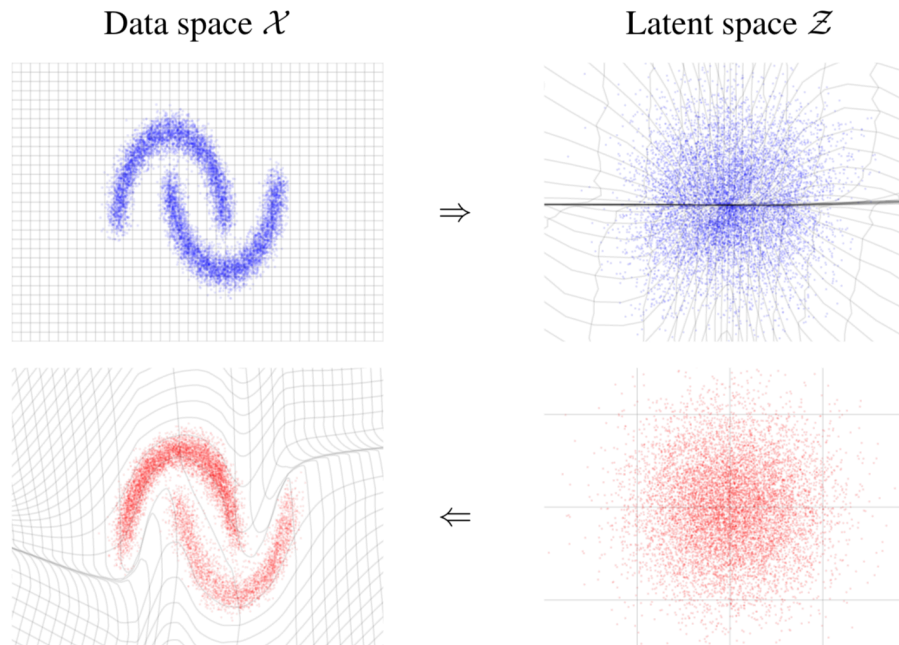
$$x \sim \hat{p}_X$$

$$z = f(x)$$

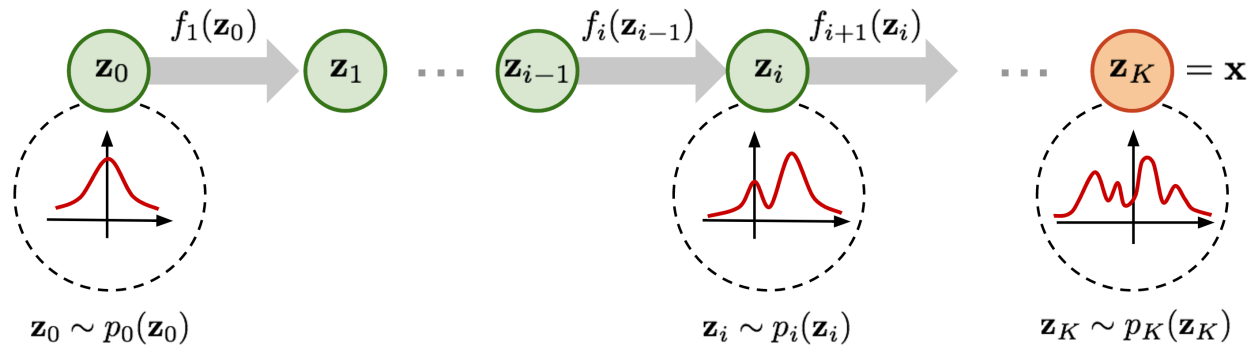
### Generation

$$z \sim p_Z$$

$$x = f^{-1}(z)$$



# Composition of Normalizing Flows



## Theorem

If every  $\{\mathbf{f}_k\}_{k=1}^K$  satisfies the conditions of the change-of-variables theorem, then the composition  $\mathbf{f}(\mathbf{x}) = \mathbf{f}_K \circ \dots \circ \mathbf{f}_1(\mathbf{x})$  also satisfies them.

$$\begin{aligned}
 p_{\theta}(\mathbf{x}) &= p(\mathbf{f}(\mathbf{x})) \left| \det \left( \frac{\partial \mathbf{f}(\mathbf{x})}{\partial \mathbf{x}} \right) \right| = p(\mathbf{f}(\mathbf{x})) \left| \det \left( \frac{\partial \mathbf{f}_K}{\partial \mathbf{f}_{K-1}} \cdots \frac{\partial \mathbf{f}_1}{\partial \mathbf{x}} \right) \right| \\
 &= p(\mathbf{f}(\mathbf{x})) \prod_{k=1}^K \left| \det \left( \frac{\partial \mathbf{f}_k}{\partial \mathbf{f}_{k-1}} \right) \right| = p(\mathbf{f}(\mathbf{x})) \prod_{k=1}^K |\det(\mathbf{J}_{\mathbf{f}_k})|
 \end{aligned}$$

# Normalizing Flows (NF)

$$\log p_{\theta}(\mathbf{x}) = \log p(\mathbf{f}_{\theta}(\mathbf{x})) + \log |\det(\mathbf{J}_{\mathbf{f}})|$$

## Definition

A normalizing flow is a  $C^1$ -diffeomorphism that transforms data  $\mathbf{x}$  to noise  $\mathbf{z}$ .

- **Normalizing** refers to mapping samples from  $p_{\text{data}}(\mathbf{x})$  to a base distribution  $p(\mathbf{z})$ .
- **Flow** describes the sequence of transformations from base noise to data.

$$\mathbf{z} = \mathbf{f}_K \circ \dots \circ \mathbf{f}_1(\mathbf{x}); \quad \mathbf{x} = \mathbf{f}_1^{-1} \circ \dots \circ \mathbf{f}_K^{-1}(\mathbf{z})$$

## Log-Likelihood

$$\log p_{\theta}(\mathbf{x}) = \log p(\mathbf{f}_K \circ \dots \circ \mathbf{f}_1(\mathbf{x})) + \sum_{k=1}^K \log |\det(\mathbf{J}_{\mathbf{f}_k})|$$

where  $\mathbf{J}_{\mathbf{f}_k} = \frac{\partial \mathbf{f}_k}{\partial \mathbf{f}_{k-1}}$ .

We focus on **continuous** variables; **discrete flows** have not become mainstream.

# Normalizing Flows

Use a fixed base distribution  $p(\mathbf{z})$ , e.g.  $\mathcal{N}(0, \mathbf{I})$ .

## Training

1. Sample a minibatch  $\mathbf{x}_i \sim p_{\text{data}}(\mathbf{x})$ ,  $i = 1, \dots, n$ .
2. Compute  $\mathbf{z}_i = \mathbf{f}_{\theta}(\mathbf{x}_i)$  and  $\log |\det \mathbf{J}_{\mathbf{f}}(\mathbf{x}_i)|$ .
3. Compute loss

$$\mathcal{L} = -\frac{1}{n} \sum_{i=1}^n [\log p(\mathbf{z}_i) + \log |\det \mathbf{J}_{\mathbf{f}}(\mathbf{x}_i)|] .$$

4. Update  $\theta$  by gradient descent on  $\mathcal{L}$ .

## Requirements

- **Training:** efficient computation of  $\log |\det \mathbf{J}_{\mathbf{f}}|$ , where  $\mathbf{J}_{\mathbf{f}} = \frac{\partial \mathbf{f}_{\theta}(\mathbf{x})}{\partial \mathbf{x}}$ .
- **Sampling:** efficient inversion of the transformation  $\mathbf{f}_{\theta}$ .

What's the computational complexity of evaluating this determinant?

## Sampling

1. Sample  $\mathbf{z} \sim p(\mathbf{z})$ .
2. Compute  $\mathbf{x} = \mathbf{f}_{\theta}^{-1}(\mathbf{z})$ .
3. Return  $\mathbf{x}$ .

# Outline

## 01 Normalizing Flows (NF)

## 02 **NF Examples**

Linear NF

Gaussian Autoregressive NF

Blockwise Flows (RealNVP / TarFlow)

## 03 Latent Variable Models (LVM)

# Outline

## 01 Normalizing Flows (NF)

## 02 NF Examples

### Linear NF

Gaussian Autoregressive NF

Blockwise Flows (RealNVP / TarFlow)

## 03 Latent Variable Models (LVM)

# Jacobian Structure

## Normalizing Flows Log-Likelihood

$$\log p_{\theta}(\mathbf{x}) = \log p(\mathbf{f}_{\theta}(\mathbf{x})) + \log \left| \det \left( \frac{\partial \mathbf{f}_{\theta}(\mathbf{x})}{\partial \mathbf{x}} \right) \right|$$

The principal computational challenge is evaluating the Jacobian determinant.

### What is $\det(\mathbf{J})$ in These Cases?

Consider a linear layer  $\mathbf{z} = \mathbf{W}\mathbf{x}$ ,  $\mathbf{W} \in \mathbb{R}^{m \times m}$ .

1.  $\mathbf{z}$  is a permutation of  $\mathbf{x}$ .
2.  $z_j$  depends only on  $x_j$ .

$$\log \left| \det \left( \frac{\partial \mathbf{f}_{\theta}(\mathbf{x})}{\partial \mathbf{x}} \right) \right| = \log \left| \prod_{j=1}^m \frac{\partial f_{j,\theta}(x_j)}{\partial x_j} \right| = \sum_{j=1}^m \log \left| \frac{\partial f_{j,\theta}(x_j)}{\partial x_j} \right|$$

3.  $z_j$  depends only on  $\mathbf{x}_{1:j}$  (autoregressive dependency).

# Linear Normalizing Flows

$$\mathbf{z} = \mathbf{f}_\theta(\mathbf{x}) = \mathbf{W}\mathbf{x}, \quad \mathbf{W} \in \mathbb{R}^{m \times m}, \quad \mathbf{J}_f = \mathbf{W}$$

Computing  $\mathbf{f}_\theta^{-1}(\mathbf{z})$  means solving  $\mathbf{W}\mathbf{x} = \mathbf{z}$ :  $O(m^3)$  for a general dense matrix.

Diagonal systems cost  $O(m)$ ; triangular systems cost  $O(m^2)$ .

## Continuous Parameterization

There is no continuous surjective map from  $\mathbb{R}^{m^2}$  onto the set of all invertible  $m \times m$  matrices.

Explanation: a continuous path from  $\det \mathbf{W} > 0$  to  $\det \mathbf{W} < 0$  must cross  $\det \mathbf{W} = 0$ , where invertibility is lost. A continuous parameterization that guarantees invertibility must therefore keep the determinant sign fixed.

## Structured Factors

Structured factorizations (e.g. LU or QR) simplify determinant and inverse calculations. Keep triangular diagonals nonzero to preserve invertibility.

**Learn the factors directly** during training.

# Outline

## 01 Normalizing Flows (NF)

## 02 NF Examples

Linear NF

**Gaussian Autoregressive NF**

Blockwise Flows (RealNVP / TarFlow)

## 03 Latent Variable Models (LVM)

# Gaussian Autoregressive Model

Consider the autoregressive model:

$$p_{\theta}(\mathbf{x}) = \prod_{j=1}^m p_{\theta}(x_j | \mathbf{x}_{1:j-1}), \quad p_{\theta}(x_j | \mathbf{x}_{1:j-1}) = \mathcal{N}(\mu_{j,\theta}(\mathbf{x}_{1:j-1}), \sigma_{j,\theta}^2(\mathbf{x}_{1:j-1}))$$

## Generative Direction

$$x_j = \sigma_{j,\theta}(\mathbf{x}_{1:j-1}) \cdot z_j + \mu_{j,\theta}(\mathbf{x}_{1:j-1}), \quad z_j \sim \mathcal{N}(0, 1)$$

## Inverse Transformation

$$z_j = \frac{x_j - \mu_{j,\theta}(\mathbf{x}_{1:j-1})}{\sigma_{j,\theta}(\mathbf{x}_{1:j-1})}$$

- This gives a  $C^1$ -**diffeomorphism** from  $p(\mathbf{z})$  to  $p_{\theta}(\mathbf{x})$  (assume that  $\sigma_j \neq 0$ ).
- This model is called an autoregressive (AR) NF with base distribution  $p(\mathbf{z}) = \mathcal{N}(0, \mathbf{I})$ .
- The Jacobian matrix of this transformation is triangular.

# Gaussian Autoregressive NF: Two Directions

## Sampling: sequential

$$x_j = \mu_{j,\theta}(\mathbf{x}_{1:j-1}) + \sigma_{j,\theta}(\mathbf{x}_{1:j-1})z_j$$

$$\mu_{j,\theta}(\mathbf{x}_{1:j-1}) = \sum_{i<j} x_i, \quad \sigma_{j,\theta} = 1 \quad \text{A fixed toy Gaussian AR flow}$$

## Density evaluation: parallel

$$z_j = \frac{x_j - \mu_{j,\theta}(\mathbf{x}_{1:j-1})}{\sigma_{j,\theta}(\mathbf{x}_{1:j-1})}$$

### Sampling: sequential

$$x_j = z_j + \sum_{i<j} x_i$$

| $x_1$  | $x_2$  | $x_3$  | $x_4$  |
|--------|--------|--------|--------|
| -1.000 | -0.500 | -0.500 | -2.500 |
| step 1 | step 2 | step 3 | step 4 |

Each coordinate needs the previous outputs.

### Density evaluation: parallel

$$z_j = x_j - \sum_{i<j} x_i$$

| $z_1$     | $z_2$     | $z_3$     | $z_4$     |
|-----------|-----------|-----------|-----------|
| -1.000    | 0.500     | 1.000     | -0.500    |
| same step | same step | same step | same step |

All prefixes are already available in the observed vector.

**Sampling: slow** — coordinates are computed sequentially.

**Density evaluation: fast** — coordinates are computed in parallel.

# Outline

## 01 Normalizing Flows (NF)

## 02 NF Examples

Linear NF

Gaussian Autoregressive NF

**Blockwise Flows (RealNVP / TarFlow)**

## 03 Latent Variable Models (LVM)

# RealNVP

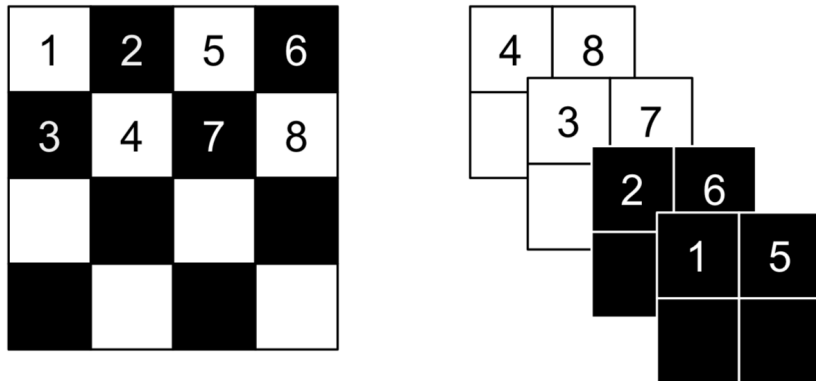
Split  $\mathbf{x}$  and  $\mathbf{z}$  into two parts:

$$\mathbf{x} = [\mathbf{x}_1, \mathbf{x}_2] = [\mathbf{x}_{1:d}, \mathbf{x}_{d+1:m}]; \quad \mathbf{z} = [\mathbf{z}_1, \mathbf{z}_2] = [\mathbf{z}_{1:d}, \mathbf{z}_{d+1:m}]$$

## Coupling Layer

$$\begin{cases} \mathbf{x}_1 = \mathbf{z}_1 \\ \mathbf{x}_2 = \mathbf{z}_2 \odot \boldsymbol{\sigma}_\theta(\mathbf{z}_1) + \boldsymbol{\mu}_\theta(\mathbf{z}_1) \end{cases} \quad \begin{cases} \mathbf{z}_1 = \mathbf{x}_1 \\ \mathbf{z}_2 = (\mathbf{x}_2 - \boldsymbol{\mu}_\theta(\mathbf{x}_1)) \odot \frac{1}{\boldsymbol{\sigma}_\theta(\mathbf{x}_1)} \end{cases}$$

## Image Partitioning



- Checkerboard ordering corresponds to masking.
- Channelwise ordering relies on splitting.

# RealNVP

## Coupling Layer

$$\begin{cases} \mathbf{x}_1 = \mathbf{z}_1 \\ \mathbf{x}_2 = \mathbf{z}_2 \odot \sigma_{\theta}(\mathbf{z}_1) + \mu_{\theta}(\mathbf{z}_1) \end{cases} \quad \begin{cases} \mathbf{z}_1 = \mathbf{x}_1 \\ \mathbf{z}_2 = (\mathbf{x}_2 - \mu_{\theta}(\mathbf{x}_1)) \odot \frac{1}{\sigma_{\theta}(\mathbf{x}_1)} \end{cases}$$

In both training and sampling, only a single forward pass is needed!

## Jacobian

$$\det \left( \frac{\partial \mathbf{z}}{\partial \mathbf{x}} \right) = \det \begin{pmatrix} \mathbf{I}_d & 0_{d \times m-d} \\ \frac{\partial \mathbf{z}_2}{\partial \mathbf{x}_1} & \frac{\partial \mathbf{z}_2}{\partial \mathbf{x}_2} \end{pmatrix} = \prod_{j=1}^{m-d} \frac{1}{\sigma_{j,\theta}(\mathbf{x}_1)}$$

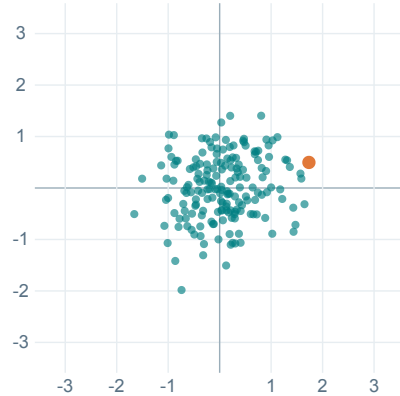
## Gaussian AR NF

$$\begin{aligned} \mathbf{x} = \mathbf{f}_{\theta}^{-1}(\mathbf{z}) &\Rightarrow \mathbf{x}_j = \sigma_{j,\theta}(\mathbf{x}_{1:j-1}) \cdot \mathbf{z}_j + \mu_{j,\theta}(\mathbf{x}_{1:j-1}) \\ \mathbf{z} = \mathbf{f}_{\theta}(\mathbf{x}) &\Rightarrow \mathbf{z}_j = (\mathbf{x}_j - \mu_{j,\theta}(\mathbf{x}_{1:j-1})) \cdot \frac{1}{\sigma_{j,\theta}(\mathbf{x}_{1:j-1})}. \end{aligned}$$

How can the RealNVP layer be derived as a special instance of the Gaussian autoregressive NF?

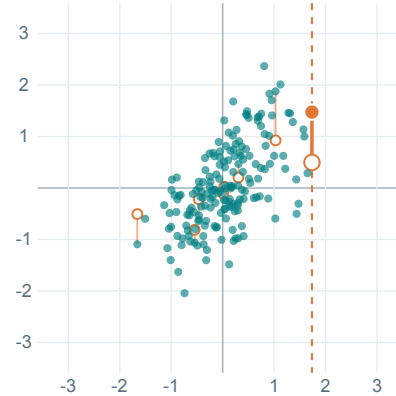
# RealNVP: Explore a Coupling Layer

Base noise



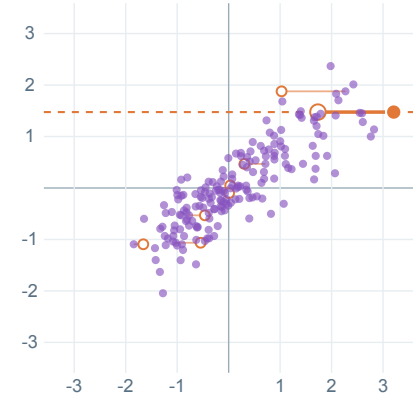
$$\mathbf{z} \sim \mathcal{N}(0, 0.65^2 \mathbf{I})$$

Layer 1: vertical motion



$$y_1 = z_1, \quad y_2 = \sigma(z_1)z_2 + \mu(z_1)$$

Layer 2: horizontal motion



$$x_2 = y_2, \quad x_1 = \sigma(y_2)y_1 + \mu(y_2)$$

$$\sigma(t) = e^{0.4\lambda \tanh(t)}, \quad \mu(t) = 0.8\lambda \tanh(t), \quad \lambda = 1.0 \quad \text{Dashed: fixed coordinate. Hollow: previous position.}$$

$$\frac{\partial \mathbf{y}}{\partial \mathbf{z}} = \begin{pmatrix} 1 & 0 \\ * & \sigma(z_1) \end{pmatrix}$$

$$\frac{\partial \mathbf{x}}{\partial \mathbf{y}} = \begin{pmatrix} \sigma(y_2) & * \\ 0 & 1 \end{pmatrix}$$

Triangular Jacobians.  
Positive, explicit scales.

The conditioner stays fixed: undo the shift, divide by the scale. For density evaluation, negate the generative log-determinant.

# TarFlow: Autoregression over Patches

## Gaussian AR NF: Autoregression

Transform one **scalar coordinate**  $x_j$ , conditioned on **all previous coordinates**  $\mathbf{x}_{1:j-1}$ .

## RealNVP: Transform a Block

Transform a **whole block**  $\mathbf{x}_2$  elementwise, with shifts and scales predicted from  $\mathbf{x}_1$ .

## TarFlow: Autoregression + Blockwise Transforms

Split an image into patches  $\mathbf{x}_j \in \mathbb{R}^d$ . Each patch is one block of pixel values.

$$\underbrace{\mathbf{x}_j}_{\text{whole patch}} = \mathbf{z}_j \odot \boldsymbol{\sigma}_{j,\theta}(\underbrace{\mathbf{x}_{1:j-1}}_{\text{previous patches}}) + \boldsymbol{\mu}_{j,\theta}(\mathbf{x}_{1:j-1}).$$

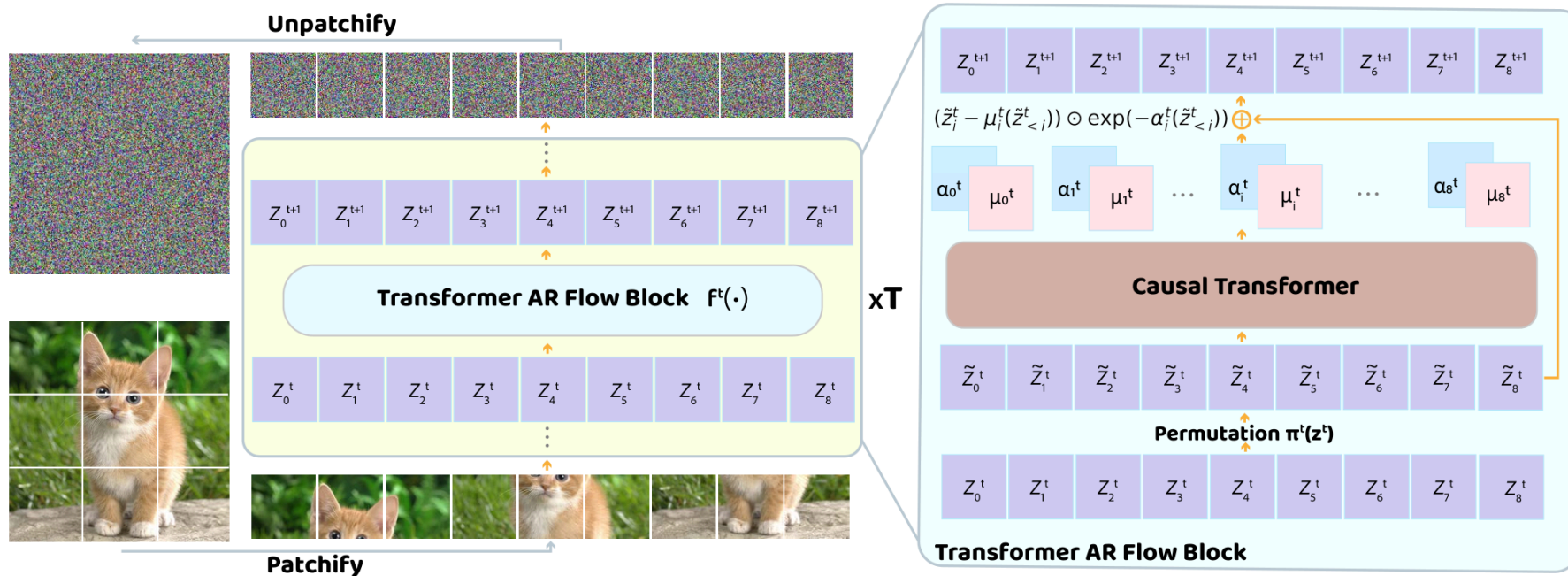
**Sampling:** sequential across patches, parallel within each patch.

**Density evaluation:** all patches are observed, so compute all  $\mathbf{z}_j$  in parallel.

The Jacobian stays **triangular**. Positive scales ensure invertibility;  $\mathbf{z}_1 = \mathbf{x}_1$ .

# TarFlow: Transformer Architecture

Use a **causal Transformer** to predict the shifts and scales of each AR flow block.



- Causal attention uses only earlier patches. The Transformer itself need not be invertible.
- Stack affine AR blocks and reverse patch order between blocks.

# Dequantization of Images

An 8-bit image has  $\mathbf{x} \in \{0, \dots, 255\}^m$ . A flow models a **continuous density**  $p_\theta(\mathbf{v})$ .

## Probability of a Pixel Bin

For a model supported on  $[0, 256)^m$ , the discrete image probability is

$$P_\theta(\mathbf{x}) = \int_{[0,1)^m} p_\theta(\mathbf{x} + \mathbf{u}) d\mathbf{u}.$$

## Uniform Dequantization

$$\mathbf{u} \sim \text{Uniform}([0, 1)^m), \quad \mathbf{v} = \mathbf{x} + \mathbf{u}, \quad \lfloor \mathbf{v} \rfloor = \mathbf{x}.$$

Each discrete value is spread over its bin. Train the flow on  $\mathbf{v}$ .

By **Jensen's inequality**,

$$\log P_\theta(\mathbf{x}) \geq \mathbb{E}_{\mathbf{u}} [\log p_\theta(\mathbf{x} + \mathbf{u})].$$

Exact continuous density evaluation gives a **lower bound** on discrete log-likelihood.

# TarFlow: Flows Can Generate Detailed Images



AFHQ  $256 \times 256$  (left); ImageNet  $128 \times 128$  and  $64 \times 64$  (right).

- The **autoregressive flow construction** scales to detailed images using networks over patches.
- The flow keeps a tractable likelihood and sequential sampling across patches.

# Normalizing Flows: What Comes Next?

We can now turn simple noise into complex data, with both **sampling** and **exact continuous density evaluation**.

## What We Learned

- Change of variables gives an exact continuous density and an MLE training objective.
- Efficient training and sampling require tractable Jacobian determinants and efficient inverses.

## Next: Latent Variable Models

Relax the one-to-one map with a probabilistic decoder. Marginalizing latent variables makes likelihood evaluation harder, motivating variational inference (L3–L4).

## Later: Continuous Flows (L9–L11)

Continuous-time normalizing flows (CNF) replace layers by an ODE, relaxing the layer constraints. Flow Matching learns the velocity field by regression; sampling uses a numerical solver.

# Outline

## 01 Normalizing Flows (NF)

## 02 NF Examples

Linear NF

Gaussian Autoregressive NF

Blockwise Flows (RealNVP / TarFlow)

## 03 Latent Variable Models (LVM)

# Bayesian Framework

## Bayes' Theorem

$$p(\boldsymbol{\theta}|\mathbf{x}) = \frac{p(\mathbf{x}|\boldsymbol{\theta})p(\boldsymbol{\theta})}{p(\mathbf{x})} = \frac{p(\mathbf{x}|\boldsymbol{\theta})p(\boldsymbol{\theta})}{\int p(\mathbf{x}|\boldsymbol{\theta})p(\boldsymbol{\theta})d\boldsymbol{\theta}}$$

- $\mathbf{x}$ : observed variables;
- $\boldsymbol{\theta}$ : unknown latent variables/parameters;
- $p_{\boldsymbol{\theta}}(\mathbf{x}) = p(\mathbf{x}|\boldsymbol{\theta})$ : likelihood;
- $p(\mathbf{x}) = \int p(\mathbf{x}|\boldsymbol{\theta})p(\boldsymbol{\theta})d\boldsymbol{\theta}$ : evidence;
- $p(\boldsymbol{\theta})$ : prior distribution;
- $p(\boldsymbol{\theta}|\mathbf{x})$ : posterior distribution.

## Interpretation

The prior  $p(\boldsymbol{\theta})$  expresses our belief before observing data; the posterior  $p(\boldsymbol{\theta}|\mathbf{x})$  updates it after observing  $\mathbf{x}$ .

## Computational Challenge

The evidence requires integration over the unobserved variables. If it is intractable, the posterior cannot be evaluated exactly.

# Latent Variable Models (LVM)

## Maximum Likelihood Estimation (MLE) Problem

For the observed dataset  $\mathbf{X} = \{\mathbf{x}_i\}_{i=1}^n$ ,

$$\boldsymbol{\theta}^* = \arg \max_{\boldsymbol{\theta}} p_{\boldsymbol{\theta}}(\mathbf{X}) = \arg \max_{\boldsymbol{\theta}} \prod_{i=1}^n p_{\boldsymbol{\theta}}(\mathbf{x}_i) = \arg \max_{\boldsymbol{\theta}} \sum_{i=1}^n \log p_{\boldsymbol{\theta}}(\mathbf{x}_i).$$

The distribution  $p_{\boldsymbol{\theta}}(\mathbf{x})$  should be highly complex (just like the true data distribution  $p_{\text{data}}(\mathbf{x})$ ).

## Latent Variable

Introduce a latent variable  $\mathbf{z}$  for each observed sample  $\mathbf{x}$ :

$$p_{\boldsymbol{\theta}}(\mathbf{x}, \mathbf{z}) = p_{\boldsymbol{\theta}}(\mathbf{x}|\mathbf{z})p(\mathbf{z}); \quad \log p_{\boldsymbol{\theta}}(\mathbf{x}, \mathbf{z}) = \log p_{\boldsymbol{\theta}}(\mathbf{x}|\mathbf{z}) + \log p(\mathbf{z}).$$

$$p_{\boldsymbol{\theta}}(\mathbf{x}) = \int p_{\boldsymbol{\theta}}(\mathbf{x}, \mathbf{z})d\mathbf{z} = \int p_{\boldsymbol{\theta}}(\mathbf{x}|\mathbf{z})p(\mathbf{z})d\mathbf{z}.$$

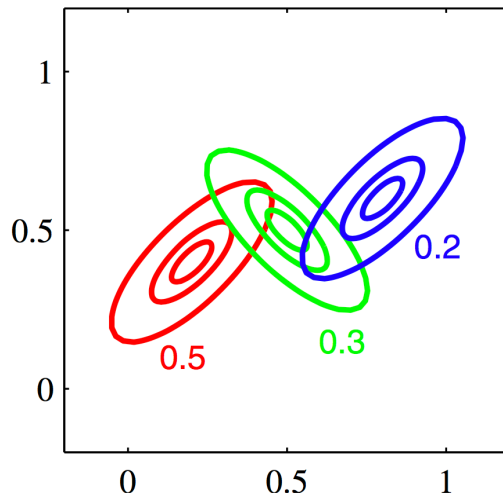
- $p(\mathbf{z})$  is the **prior** and  $p_{\boldsymbol{\theta}}(\mathbf{x}|\mathbf{z})$  is the **decoder** distribution.
- Both  $p_{\boldsymbol{\theta}}(\mathbf{x}|\mathbf{z})$  and  $p(\mathbf{z})$  are usually much simpler than  $p_{\boldsymbol{\theta}}(\mathbf{x})$ .

# Latent Variable Models (LVM)

$$\log p_{\theta}(\mathbf{x}) = \log \int p_{\theta}(\mathbf{x}|\mathbf{z})p(\mathbf{z})d\mathbf{z} \rightarrow \max_{\theta}$$

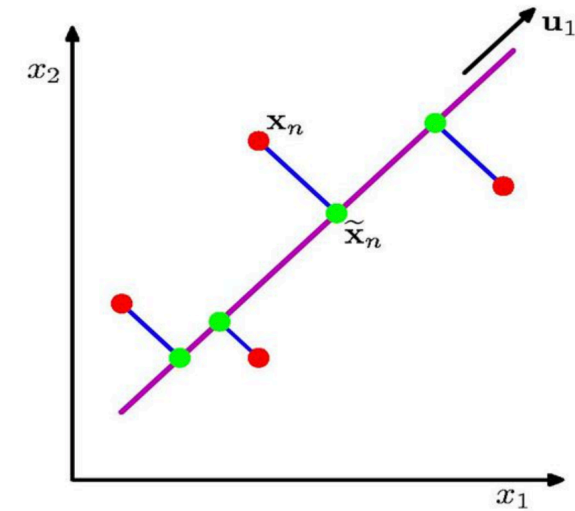
## Examples

### Mixture of Gaussians



- $p_{\theta}(\mathbf{x}|\mathbf{z}) = \mathcal{N}(\boldsymbol{\mu}_z, \boldsymbol{\Sigma}_z)$
- $p(\mathbf{z}) = \text{Cat}(\boldsymbol{\pi})$

### PCA Model



- $p_{\theta}(\mathbf{x}|\mathbf{z}) = \mathcal{N}(\mathbf{W}\mathbf{z} + \boldsymbol{\mu}, \sigma^2\mathbf{I})$
- $p(\mathbf{z}) = \mathcal{N}(0, \mathbf{I})$

# Summary

- Change of variables computes exact continuous densities under invertible transformations.
- Normalizing flows use invertible maps with tractable Jacobians; CNF and Flow Matching extend this view of transporting distributions.
- Linear NFs use structured matrix factors to simplify determinant and inverse calculations.
- TarFlow uses Transformers to scale Gaussian AR flows with triangular Jacobians.
- RealNVP coupling is a special case of AR NF with fast density evaluation and sampling.
- Uniform dequantization gives a lower bound on discrete image log-likelihood.
- LVMs combine a decoder  $p_{\theta}(\mathbf{x}|\mathbf{z})$  and prior  $p(\mathbf{z})$ ; marginalization makes  $p_{\theta}(\mathbf{x})$  harder to evaluate.